Strategy-Relative Regime Switching as a Verifiable RLVR Environment for Financial Planning

Anonymous Authors

Submitted to NeurIPS 2026

Abstract

We introduce the first verifiable reinforcement learning (RLVR) environment for financial planning, designed for large language model (LLM) post-training. The environment is grounded in the strategy-relative regime switching framework of Bilokon (2026), where an agent must make binary switching decisions under an observable but noisy signal, subject to switching costs. Bellman backward induction on a discretized state space provides exact optimal policies, enabling mathematically rigorous reward computation. We conduct a parameter landscape analysis across 1,500 configurations, identifying the regions where forward-looking planning diverges from myopic (greedy) behavior. Signal persistence (κ) emerges as the dominant parameter: slow mean-reversion ($\kappa = 0.1$) yields up to 22% state-level disagreement between optimal and greedy policies, while fast mean-reversion ($\kappa = 0.7$) reduces this to 1.2%. We propose a two-bucket evaluation methodology that separates "easy" decisions (where greedy coincides with optimal) from "hard" decisions (where they diverge), extracting maximal diagnostic information from limited API calls. Evaluating Claude Opus 4 on 30 instances (750 decisions, of which 682 are easy and 68 are hard after correcting a parser bug), we find 99.3% accuracy on easy decisions and a 5.9% planning rate on hard decisions — the model matches greedy behavior on 94.1% of hard decisions, showing near-zero evidence of forward-looking reasoning. CoT inspection of all saved responses confirms 2/68 (2.9%) genuine planning instances. The gym is open-source with single-function TRL integration.

1. Introduction

Verifiable reinforcement learning from verifiable rewards (RLVR) has emerged as a powerful paradigm for LLM post-training. DeepSeek-R1 (DeepSeek-AI et al., 2025) demonstrated that GRPO training with verifiable rewards can elicit chain-of-thought reasoning in mathematical problem-solving. Reasoning Gym (Stojanovski et al., 2025) extended this to a diverse suite of puzzles with computable solutions. These successes share a common requirement: the existence of a deterministic, computable ground truth against which model outputs can be scored.

However, existing RLVR gyms test a specific and narrow type of reasoning: **static problem-solving with complete information**. A math problem presents all its premises at once, and the model produces a single answer. A coding problem presents a specification, and the model produces a single program. There is no unfolding situation, no uncertainty about future inputs, and no cost to changing one's mind.

Many real-world capabilities require a fundamentally different type of reasoning: **sequential decision-making under uncertainty**, where the agent must act before the full picture is revealed, and where each action constrains or enables future actions. Consider the contrast:

> **Math gym (static):** "A trader pays a fee of \$0.15 to enter or exit a position. The expected daily return is 0.30 times the signal. The signal is +0.35. Should the trader enter?"
>
> A single computation: $0.30 \times 0.35 = 0.105 < 0.15$. Answer: no.
>
> **Our gym (sequential):** The same trader, same parameters — but the signal was +0.35 yesterday, is +0.35 today, and will be revealed tomorrow. If the trader enters today, they pay \$0.15 once and earn returns for as long as the signal stays positive. If the signal is persistent (slow mean-reversion), the cumulative returns over many days will exceed the one-time entry cost. The correct answer depends on *how long the signal is likely to persist* — information that requires reasoning about the dynamics of the process, not just the current snapshot.

This distinction — reasoning about a single state versus reasoning about a trajectory through states — is what separates static from sequential planning. Existing RLVR gyms do not test this capability.

Financial planning provides a natural domain for sequential reasoning under uncertainty. A portfolio manager deciding whether to enter a trade must weigh the immediate expected profit against the cost of entry, the probability that the opportunity persists, and the cost of exiting if conditions change. This multi-step, cost-aware reasoning under partial information is precisely the capability we aim to train and evaluate.

The verification challenge. Constructing a verifiable gym for sequential financial planning faces a fundamental obstacle: most financial problems lack computable optimal solutions. Portfolio optimization depends on unknown return distributions (Merton, 1969). Optimal execution requires

calibrated market impact models (Almgren and Chriss, 2001). Reinforcement learning environments for finance, such as FinRL (Liu et al., 2022), rely on simulation-based evaluation rather than analytic ground truth, making them unsuitable for RLVR.

We resolve this by building on Bilokon (2026), who defines strategy-relative market regimes as filtration compressions of the full market information and derives optimal binary regime processes via Bellman dynamic programming. In this framework, an agent observes a scalar signal Z_t that follows a known stochastic process, and must decide at each step whether to be "on" ($s_t = 1$) or "off" ($s_t = 0$), incurring a switching cost λ each time the regime changes. Because the signal process is fully specified and the state space is finite (after discretization), the Bellman recursion can be solved exactly via backward induction, providing a computable optimal policy. The gym thus tests sequential reasoning under uncertainty with a verifiable reward — a combination that no existing RLVR environment offers.

Contributions.

1. We present the first verifiable RLVR environment for sequential financial planning, with exact ground-truth solutions computed via Bellman backward induction. Unlike math or code gyms, the environment requires multi-step decisions under uncertainty where actions have path-dependent consequences.
2. We characterize the parameter landscape across 1,500 configurations, identifying when and why forward-looking planning differs from greedy (myopic) behavior.
3. We introduce a two-bucket evaluation methodology that decomposes LLM performance into comprehension (can the model evaluate immediate payoffs?) and planning (can the model override locally attractive actions when the future makes them suboptimal?).
4. We evaluate a frontier LLM (Claude Opus 4) and find near-perfect comprehension (99.3% easy accuracy) but near-zero planning (5.9% hard accuracy, 2.9% confirmed by CoT inspection), providing a concrete baseline for future RLVR training. The comprehension gap previously reported (77.4%) was entirely due to a parser bug and does not reflect genuine model failures; the model shows essentially no evidence of sequential planning beyond myopic cost-benefit analysis.

---

2. Problem Formulation

2.1 Strategy-Relative Regime Switching

Following Bilokon (2026), we consider a filtered probability space $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}_{t=0}^{T}, \mathbb{P})$ with an observable signal process Z_t and a PnL process X_{t+1}. The agent controls a binary regime process $s_t \in \{0, 1\}$, adapted to the filtration generated by $(Z_0, \ldots, Z_t, s_{t-1})$.

The objective is to maximize the expected net utility:

$$J(s) = \mathbb{E}\left[\sum_{t=0}^{T-1} u(s_t \cdot X_{t+1}) - \lambda \cdot \mathbf{1}\{s_t \neq s_{t-1}\}\right]$$

where $u(\cdot)$ is the utility function (we use the identity, $u(x) = x$), $\lambda > 0$ is the switching cost, and $s_{-1} = 0$ by convention.

2.2 Bellman Recursion

Define the value function $V_t(z, s^-)$ as the optimal expected future payoff from time t onward, given current signal z and previous regime s^-:

$$V_t(z, s^-) = \max_{a \in \{0,1\}} \left[u(a \cdot \alpha \cdot z) - \lambda \cdot \mathbf{1}\{a \neq s^-\} + \int V_{t+1}(z')\, P(dz'|z) \right]$$

with terminal condition $V_T(z, s^-) = 0$. The optimal policy at each state is the arg max of this expression.

2.3 Data Generating Process

We specify Z_t as an Ornstein-Uhlenbeck (OU) process:

$$dZ_t = \kappa(\theta - Z_t)\, dt + \sigma_z\, dW_t$$

with $\theta = 0$ (zero long-run mean). The PnL conditional on the signal is:

$$X_{t+1}|Z_t \sim \mathcal{N}(\alpha \cdot Z_t,\, \sigma_x^2)$$

where $\alpha > 0$ controls signal strength and σ_x^2 is idiosyncratic noise. The parameter $\kappa > 0$ controls mean-reversion speed: small κ yields persistent signals (favorable for planning), while large κ yields rapidly decaying signals (greedy is near-optimal).

2.4 Numerical Solution

The Bellman equation is solved via backward induction on a discretized z-grid. The transition expectation $\int V_{t+1}(z')P(dz'|z)$ is computed using Gauss-Hermite quadrature, exploiting the Gaussian transition density of the OU process. This yields an exact (up to discretization error) optimal policy $\pi^*(z, s^-)$ for every grid point and time step.

---

3. Gym Architecture

The gym is structured around three components: a **Generator**, a **Verifier**, and a **Prompt Manager**.

3.1 Generator

The Generator creates problem instances by:

1. Sampling parameters $(\kappa, \lambda, \alpha, \sigma_z, T)$ from a configured distribution or grid.
2. Solving the Bellman recursion on the discretized state space to obtain the optimal policy π^*.
3. Simulating signal trajectories $\{Z_t\}_{t=0}^{T}$ from the OU process.
4. Computing the optimal, greedy, and random agent decisions and expected payoffs on each trajectory.

Multi-seed baselines (default: 50 seeds per configuration) provide robust expected-value estimates for regret normalization.

3.2 Verifier

The Verifier parses the LLM's decision at each time step, computes the realized payoff, and produces a regret-normalized score:

$$\text{score} = \frac{J_{\text{LLM}} - J_{\text{random}}}{J_{\text{optimal}} - J_{\text{random}}}$$

This normalization maps random performance to 0 and optimal performance to 1. Scores above 1.0 are possible on individual instances because the Bellman solution optimizes in expectation, not for each realization. Degenerate instances where $J_{\text{optimal}} \approx J_{\text{random}}$ are handled by a minimum-denominator threshold to prevent numerical instability, and scores are clipped to $[-1, 2]$.

3.3 Prompt Manager and Information Constraint

The multi-turn conversation protocol is central to the gym's validity. At each time step t, the LLM receives:

- The current signal value Z_t (rounded to 2 decimal places).
- Its previous decision s_{t-1}.
- The PnL realized from the previous step.
- A reminder of the switching cost λ and signal coefficient α.

Critically, the LLM does **not** receive future signal values. This enforces the information constraint $s_t \in \sigma(Z_0, \ldots, Z_t, s_{t-1})$, matching the filtration under which the Bellman solution is optimal.

3.4 TRL Integration

The gym exposes a single reward function compatible with TRL's GRPO trainer:

```python
def reward_fn(completions: list[str], problems: list[dict]) -> list[float]:
    ...
```

This enables direct integration into RLVR training pipelines without modification to the training loop.

3.5 Extensibility

The Generator, Verifier, and Prompt Manager are implemented as abstract base classes, allowing the suite to be extended with additional problem types (e.g., multi-asset switching, continuous action spaces) while preserving the evaluation infrastructure.

---

4. Parameter Landscape Analysis

4.1 Sweep Design

We conduct a full factorial sweep over:

Parameter	Values	Count
κ (mean-reversion speed)	0.1, 0.2, 0.3, 0.5, 0.7	5
λ (switching cost)	0.05, 0.10, 0.15, 0.20, 0.30	5
α (signal strength)	0.10, 0.15, 0.20, 0.25, 0.30	5
σ_z (signal volatility)	0.1, 0.2, 0.3	3
T (horizon)	5, 10, 15, 25	4

This produces $5 \times 5 \times 5 \times 3 \times 4 = 1{,}500$ configurations, each solved to exact optimality.

4.2 Disagreement Metric

For each configuration, we compute a **solver-based disagreement metric**: the fraction of (z, s^-) states at which the Bellman optimal policy differs from the greedy (myopic) policy, weighted by the stationary distribution of the OU process. This metric quantifies how much planning matters for a given parameter set, independent of any LLM evaluation.

4.3 Key Findings

Signal persistence dominates. The mean-reversion speed κ is the single most important parameter (Figure D). At $\kappa = 0.1$ (slow mean-reversion, persistent signals), the average disagreement across all other parameter combinations is 13.1%. At $\kappa = 0.7$ (fast mean-reversion), average disagreement drops to 1.2%. This is intuitive: when the signal is persistent, the current observation is informative about future payoffs, making forward-looking behavior valuable. When the signal reverts quickly, the future is approximately independent of the present, and greedy is near-optimal.

The λ/α ratio has a non-monotone effect. Disagreement peaks at $\lambda/\alpha \in [0.33, 0.50]$ and falls at both extremes. When λ/α is very small, switching is cheap and both optimal and greedy switch frequently. When λ/α is very large, switching is expensive and both policies rarely switch. It is the intermediate regime that creates the largest gap between forward-looking and myopic behavior.

Horizon is a moderate amplifier. Longer horizons ($T = 25$) produce modestly higher disagreement than shorter ones ($T = 5$), because the value of planning accumulates over more steps.

Disagreement ceiling. The maximum observed disagreement is approximately 22% of states on the discretized grid (Figure A). On actual simulated trajectories, approximately 10% of visited decisions differ between optimal and greedy. This ceiling reflects the inherent structure of the problem: the optimal and greedy policies agree in most of the state space, differing only in a bounded "disagreement zone."

The top configurations by disagreement are reported in Appendix A.

---

5. The Planning Mechanism

5.1 Greedy vs. Optimal Thresholds

To understand *how* optimal behavior differs from greedy, we analyze the switching thresholds.

Greedy policy. The greedy (myopic) agent switches from OFF to ON when the immediate expected gain exceeds the switching cost:

$$\alpha \cdot z > \lambda \implies z > \frac{\lambda}{\alpha}$$

For our evaluation configuration ($\lambda = 0.15$, $\alpha = 0.30$), this gives a greedy threshold of $z^{\text{greedy}}_{\text{OFF}\to\text{ON}} = 0.50$.

Optimal policy. The optimal agent switches from OFF to ON at a *lower* threshold. At our evaluation configuration, we observe $z^{\text{opt}}_{\text{OFF}\to\text{ON}} \approx 0.22$, verified by manual computation of Q-values from the Bellman solution.

5.2 Why Optimal is More Aggressive

The optimal policy is more aggressive (switches on sooner) because it amortizes the one-time switching cost λ over the persistent future gains. When κ is small, a positive signal $Z_t = 0.35$ is likely to remain positive for many future steps. The greedy agent sees that $\alpha \cdot 0.35 = 0.105 < \lambda = 0.15$ and stays off. The optimal agent recognizes that the cumulative expected gain from switching on now — across multiple future steps where the signal remains positive — exceeds the one-time cost.

Formally, the Q-value difference at the switch point satisfies:

$$Q(z, a{=}1, s^{-}{=}0) - Q(z, a{=}0, s^{-}{=}0) > 0 \quad \text{for } z \in [0.22, 0.50]$$

This was verified by direct computation of Q-values at $(t = 7, z = 0.35)$ from the Bellman table (Appendix D).

5.3 The Disagreement Zone

The disagreement zone is the set of signal values where optimal and greedy policies differ (Figure C):

$$\mathcal{D}_t = \{z : \pi_t^*(z, s^-) \neq \pi_t^{\text{greedy}}(z, s^-)\}$$

For the OFF → ON direction: $\mathcal{D} \approx [0.22, 0.50]$. By symmetry, for the ON → OFF direction: $\mathcal{D} \approx [-0.50, -0.22]$.

At the terminal step ($t = T - 1$), the optimal policy reduces to greedy exactly, since there is no future to plan for. The disagreement zone widens as we move backward in time, reaching its maximum at early time steps (Figure C).

5.4 Hysteresis

Both policies exhibit hysteresis: the ON → OFF threshold differs from the OFF → ON threshold. For the optimal policy, the ON → OFF threshold is approximately -0.22 (symmetric with the OFF → ON threshold around zero). For the greedy policy, it is -0.50.

---

6. LLM Evaluation

6.1 Setup

We evaluate Claude Opus 4 (`claude-opus-4-20250514`) on 30 instances at the configuration ($\kappa = 0.1, \lambda = 0.15, \alpha = 0.30, T = 25$), producing 750 total decisions. Each instance requires 25 sequential API calls (one per time step), for a total of 750 API calls.

6.2 Two-Bucket Methodology

We partition decisions into two buckets based on the *solver's* policies, not the LLM's:

- **Easy decisions** ($n = 682$): States where the greedy and optimal policies agree. Any agent that correctly understands the immediate payoff structure should get these right.
- **Hard decisions** ($n = 68$): States where the greedy and optimal policies disagree — the disagreement zone. Only a forward-looking agent should match the optimal policy on these.

This partition is computed entirely from the solver, requiring no additional LLM queries. It extracts maximum diagnostic information from limited evaluation budgets by separating two distinct capabilities: comprehension (understanding the problem) and planning (reasoning about the future).

Note on bucket counts. The original evaluation reported $n_{\text{easy}} = 625$ and $n_{\text{hard}} = 125$. These figures were produced by a parser that failed to match the `s_t = 1` decision format, causing genuine ON decisions to be misread as 0. The corrected counts ($n_{\text{easy}} = 682$, $n_{\text{hard}} = 68$) reflect the fixed parser; the raw text of all 750 responses is archived for independent verification.

6.3 Temporal Distribution of Hard Decisions

Hard decisions are not concentrated at the beginning of the episode — they are spread across the full trajectory. Among the 68 hard decisions in the corrected Opus evaluation:

Episode phase	Prior signals seen	Share of hard decisions
Early ($t = 0$–2)	0–2	15%
Mid ($t = 3$–15)	3–15	53%
Late ($t = 16$+)	16+	32%

The average and median hard decision occurs at $t = 11$, meaning the model has already observed approximately 11 prior signal values when it faces a decision that requires forward-looking reasoning.

This is a meaningful quantity of history. With 11 prior observations of an OU process, the agent has sufficient data to form an empirical estimate of signal persistence. Yet Opus follows greedy logic on 94.1% of hard decisions, including those encountered at mid-episode where the history is abundant.

This rules out a simple information-deficit explanation: the model is not failing because it lacks data about the signal process. The failure persists even when the signal history is rich enough to potentially infer κ. This suggests a genuine reasoning gap — the model does not perform the inference from observed signal autocorrelation to future persistence, and even if it did, it does not translate that persistence estimate into a forward-looking switching decision. The gap is in dynamic reasoning, not in information availability.

6.4 Results

Easy bucket. Opus correctly matches the optimal (= greedy) policy on 677 of 682 easy decisions, yielding an accuracy of **99.3%**. The near-perfect easy accuracy confirms that the model understands the immediate payoff structure: the earlier reported 77.4% was entirely an artifact of a parser bug that misread ON decisions as 0 (see Section 8 and Appendix B for details).

Hard bucket. Of 68 hard decisions, Opus matches the optimal policy on **4** (5.9%) and matches the greedy policy on **64** (94.1%). Zero decisions match neither. This is the central finding: on decisions where planning matters, Opus behaves almost identically to a greedy agent.

CoT verification. Manual inspection of the chain-of-thought for all 68 hard decisions found 2 instances of genuine planning (2.9%) and 2 borderline instances (2.9%, near-threshold arithmetic that could be interpreted as planning):

- *Seed 14, t=2* (Z=+0.31, margin=0.057): Model cited "increasing positive momentum", "23 steps remaining", "expected value of switching ON is positive."
- *Seed 28, t=13* (Z=+0.40, margin=0.031): Model tracked "missed 0.25 in potential profits exceeds switching cost", "12 steps remaining and evidence of strong positive signals."

The remaining 64 hard decisions show no evidence of forward-looking reasoning in the CoT.

J-value comparison. Mean cumulative payoff (J) across 30 instances:

Agent	Mean J
Optimal (Bellman)	+1.30
Greedy	+1.19
Opus	−0.16
Random	−1.81

Statistical tests (two-sample t-tests, $N = 30$):

Comparison	p-value	Conclusion
Opus vs. Random	< 0.0001	Opus > Random
Opus vs. Greedy	$= 0.0001$	Opus < Greedy
Greedy vs. Optimal	$= 0.82$	Not significant

Opus significantly outperforms random but significantly underperforms greedy. The Greedy vs. Optimal comparison is not significant at $N = 30$; this is expected given the small per-instance J difference (+1.19 vs. +1.30) and is confirmed by separate Goldilocks analysis at $N = 500$ instances.

6.5 Failure Modes

Manual inspection of Opus transcripts reveals two qualitatively distinct failure modes:

1. **Pathological alternation.** On some episodes (e.g., seed 8; Figure F), Opus switches regime at nearly every step, incurring 25 switching costs in 25 decisions. This suggests the model is reacting to noise rather than signal, and does not internalize that switching has a cost that should deter frequent changes.
2. **Excessive caution.** On other episodes, Opus never switches on, missing all positive-signal opportunities. This appears to result from over-weighting the switching cost relative to cumulative gains.

Both failure modes are consistent with the absence of forward-looking reasoning: the model does not plan over the multi-step horizon to amortize costs against persistent gains.

6.6 Kappa Inference Test

To test whether Opus adapts to different signal dynamics, we conducted a controlled comparison: two sets of 5 episodes with identical known parameters ($\alpha = 0.30$, $\lambda = 0.15$, $T = 15$) but different unknown mean-reversion speeds ($\kappa = 0.1$ vs. $\kappa = 0.7$). The model received the same prompt format in both conditions and could only distinguish the regimes through the observed signal patterns.

Results: Opus exhibited identical behavior in both conditions — 95.9% easy accuracy and 0.4 mean switches per episode in each. However, only 3 hard decisions occurred across all 10 episodes (2

at $\kappa = 0.1$, 1 at $\kappa = 0.7$), making the test inconclusive for the planning dimension. The behavioral fingerprint (identical switching rate, identical easy accuracy) is consistent with the model applying the same greedy framework regardless of signal dynamics, but the small number of hard decisions prevents a definitive conclusion.

This test was designed to isolate κ inference from α/λ adaptation (which earlier tests showed the model does correctly). A definitive test would require more instances with parameter configurations that produce more hard decisions.

---

7. Discussion

7.1 The Single Planning Gap

The two-bucket analysis reveals that Opus's underperformance relative to greedy arises from a single source:

Planning gap (94.1% greedy-like on hard decisions): On the 68 decisions where forward-looking reasoning is required, Opus follows greedy logic almost perfectly. There is no evidence that the model spontaneously reasons about future signal persistence or amortizes switching costs over time.

The "comprehension gap" was a parser artifact. An earlier analysis reported 77.4% easy accuracy (484/625), implying a 22.6% comprehension failure rate. This was entirely caused by a parser bug that failed to match the `s_t = 1` decision format, causing genuine ON decisions to be counted as errors. With the fixed parser, easy accuracy is 99.3% (677/682) — near-perfect. The model correctly evaluates immediate payoffs; the only genuine gap is in dynamic planning. A parser bug that inflated error rates and created a spurious comprehension gap was discovered through manual CoT inspection and corrected (see Section 8 for a full discussion).

7.1.1 Reasoning Quality vs. Reasoning Scope

A manual step-by-step test provided an important qualification to the planning gap finding. When Opus is walked through a single hard decision interactively, its reasoning is qualitatively sophisticated: the model tracks cumulative savings from prior steps, computes round-trip switching costs, notes the remaining horizon, and articulates trade-offs between immediate cost and future opportunity. The model is not computationally incoherent — it performs the static calculations correctly. The failure is specifically on the dynamic question: when the immediate cost-benefit of greedy favors one action but the persistence of the signal over the remaining horizon favors another, the model cannot override its greedy framework. Presented with a persistent positive signal in the disagreement zone ($z \approx 0.35$, current state OFF), Opus consistently stays OFF because $\alpha \cdot z = 0.105 < \lambda = 0.15$, even when the remaining horizon is long and the signal is demonstrably slow-reverting. The gap is not in static computation or in the quality of local reasoning — it is in the ability to perform dynamic programming reasoning: recognizing that the value of switching ON must include the expected future gains, not merely the immediate step gain. This distinction matters for RLVR training: the

goal is not to improve static payoff calculation but to elicit the specific capability of multi-step amortization reasoning.

7.2 Prompt Scaffolding and Latent Planning Capability

A follow-up manual test complicates the picture without improving it.

After the API evaluation (Section 6), we conducted two additional tests of Claude Opus 4 on the same gym configuration ($\kappa = 0.1$, $\lambda = 0.15$, $\alpha = 0.30$, $T = 25$):

Test 1 — API evaluation format (Section 6): The prompt provides the rules, current signal, current state, and asks for a decision. The model must independently compute option values, notice signal trends, and reason about persistence. Result: 5.9% planning rate on hard decisions ($N = 30$ instances, 68 hard decisions after parser correction).

Test 1.5 — Manual minimal-prompt test (seed = 49): Before the scaffolded test, we conducted a step-by-step manual test using the standard (non-scaffolded) prompt on seed = 49 ($\kappa = 0.1$, $\lambda = 0.15$, $\alpha = 0.30$, $T = 10$). This confirmed the API evaluation findings at the individual level. The model achieved 9/9 on easy decisions (100%) but 0/1 on the single hard decision at $t = 0$. At $t = 0$, the model correctly computed the immediate cost-benefit (expected loss \$0.079 < \$ switching cost 0.15) and concluded "not worth switching" — the greedy answer. It explicitly noted "one mildly negative signal isn't enough evidence of a persistent trend," demonstrating rational uncertainty about signal dynamics. At $t = 1$, after seeing two consecutive negative signals, it inferred "this suggests persistence in the negative direction" and switched OFF — matching greedy and optimal. The model's reasoning was sophisticated but bounded by its one-step-at-a-time cost-benefit framework.

Test 2 — Scaffolded prompt (manual test, seed = 47): The prompt was augmented to pre-compute Option A and Option B with exact numbers, explicitly highlight the trend ("6 consecutive steps trending up"), include the instruction "Consider whether the signal is likely to persist," and display the number of steps remaining. Result: Opus matched the optimal policy on 25 of 25 decisions, including the one hard decision in that episode (at $t = 8$, the model switched ON at $Z = +0.35$, below the greedy threshold of 0.50). The model's reasoning explicitly cited signal persistence and amortization.

Importantly, the corrected results change the interpretation of this comparison. Easy accuracy with the standard prompt is already 99.3% — the scaffold added negligible benefit on easy decisions (the model already comprehended the payoff structure correctly). The entire effect of the scaffold was on the single hard decision: the scaffold performed the forward-looking reasoning the model cannot do spontaneously. The 5.9% vs. 100% comparison on hard decisions should not be read as evidence that the model can plan when prompted correctly. The scaffolded prompt essentially performed the planning for the model: it pre-computed the arithmetic, identified the pattern the model needed to notice, and directly suggested the reasoning strategy. Passing the scaffolded test is analogous to answering an exam question when the answer sheet is included in the question. The model followed the scaffold; it did not plan independently.

A real trader receives raw data. Pre-computing option values, flagging signal trends, and explicitly prompting persistence reasoning are not available at inference time in any realistic setting. The scaffolded prompt eliminates the cognitive steps that define the planning task.

The honest interpretation of this experiment is therefore the reverse of an encouraging one: the fact that a heavily scaffolded prompt is required to elicit correct behavior on even a single hard decision confirms that the planning capability is not spontaneously available. It is latent in the sense that the model can follow a planning scaffold when one is provided, but this is a weak form of capability. The model cannot construct the scaffold itself.

This has a direct implication for prompt design in the gym. The evaluation prompt should provide clear, unambiguous rules — the payoff structure, switching cost, signal coefficient — but must not scaffold the reasoning. A prompt that pre-computes options, highlights trends, or suggests persistence reasoning trivializes the gym: any model capable of following instructions will pass. The current API evaluation prompt (clear rules, no reasoning scaffold) is the appropriate level. The scaffolded prompt reveals latent capability but is unsuitable for evaluation or for RLVR training, because it removes the very reasoning steps the training is intended to produce.

For RLVR, this suggests a concrete training objective: internalize the scaffold. The model must learn, without being told, to compute option values, track signal autocorrelation, and translate persistence estimates into forward-looking switching decisions. The scaffolded prompt can serve as a behavioral target — describing what the model should eventually do spontaneously — but it cannot serve as the training prompt, because it removes the reasoning gap that RLVR is supposed to close.

7.3 Implications for RLVR Training

The near-zero planning rate on hard decisions suggests that forward-looking financial reasoning is not an emergent capability of current frontier LLMs, even with careful prompting. This provides a concrete, measurable target for RLVR training. The gym's exact ground truth and regret-normalized scoring make it directly compatible with GRPO and related policy gradient methods.

The two-bucket methodology offers a training diagnostic: hard-bucket planning rate tracks the emergence of genuine forward-looking reasoning. Easy-bucket accuracy starts near-ceiling (99.3%) and is unlikely to move substantially with RLVR training, since the comprehension capability is already present. The planning rate is the primary metric to monitor.

7.4 Relationship to Existing Work

FinRL (Liu et al., 2020) provides simulation-based RL environments for portfolio management, order execution, and market making, but lacks computable optimal solutions and therefore cannot serve as an RLVR gym. Almgren and Chriss (2001) derive closed-form optimal execution strategies, but in a continuous setting without the discrete decision structure needed for LLM evaluation. Our work complements these by providing a minimal but exactly solvable problem that isolates the planning capability.

Compared to mathematical RLVR gyms (Reasoning Gym, GSM8K), our environment tests a different cognitive skill: sequential decision-making under uncertainty with inter-temporal trade-offs. Math problems are typically single-shot; our environment requires 25 sequential decisions where each decision affects future option value.

---

8. Limitations

We aim for full transparency about the current limitations of this work.

Limited structural diversity. The gym currently contains a single problem type: binary switching on a scalar OU signal. This is far less diverse than mathematical reasoning gyms, which may contain hundreds of problem templates. While the base class design supports extension, we have not yet implemented additional problem types.

Low planning advantage ceiling. The maximum disagreement between optimal and greedy policies is approximately 22% of states on the discretized grid and approximately 10% on actual trajectories. This means the problem may be too simple to strongly differentiate agents: even a purely greedy agent captures most of the optimal value. The Goldilocks region — where planning meaningfully improves outcomes — is narrow.

Single model evaluated. We report results for one LLM (Claude Opus 4) with the improved prompt. Other models (Sonnet, Haiku) were tested only with the earlier, broken prompt, and those results are not directly comparable. We cannot claim generality across models.

Small sample size for some comparisons. At $N = 30$ instances, the Opus vs. Greedy comparison is statistically significant ($p = 0.0001$), but the Greedy vs. Optimal comparison is not ($p = 0.82$). Distinguishing greedy from optimal on the basis of mean J would require approximately $N \sim 10{,}000$ instances, which was infeasible given API costs. The two-bucket methodology partially addresses this by examining individual decisions rather than aggregate J.

Stationary signal process. The OU process is stationary and time-homogeneous, unlike real financial markets. Non-stationary dynamics, structural breaks, and fat-tailed distributions are not captured. The gym tests reasoning about a known, well-behaved DGP; transfer to real financial decision-making is entirely unvalidated.

No training results. We provide the gym infrastructure and a baseline evaluation, but we have not conducted RLVR training (e.g., GRPO). We cannot claim that the gym will successfully train planning capabilities; we only show that the capability is currently absent and measurable.

Asymmetric κ knowledge. The current gym provides κ to the optimal agent as a known parameter but does not reveal κ to the LLM. While the temporal distribution analysis (Section 6.3) shows that hard decisions occur at mid-episode where the model has substantial signal history from which κ could in principle be inferred, the comparison is not fully apples-to-apples. A POMDP formulation in which the optimal policy is also required to infer κ from observations — rather than treating it as known — would make the performance comparison strictly fair. The current setup

likely understates the LLM's disadvantage in information terms, though the persistence of failures even at $t = 11$ (with abundant history) suggests that the gap is not primarily attributable to this asymmetry.

Prompt sensitivity. LLM performance is sensitive to prompt phrasing. Our improved prompt raised $T = 1$ accuracy from 20% to 80%. This means evaluation results are partially a function of prompt quality, not only model capability. We report both prompt versions in Appendix E.

Scores above 1.0. Because the Bellman solution optimizes expected payoff, individual realized trajectories may yield scores above 1.0 when the LLM happens to benefit from favorable noise. This does not indicate super-optimal planning.

Parser bug and the spurious comprehension gap. An initial version of this evaluation used a parser that failed to match the `s_t = 1` decision format (matching `s_t = 0` correctly but over-matching step identifiers and state labels, causing ON decisions to be silently dropped). This inflated the apparent error rate on easy decisions from 0.7% to 22.6%, creating a spurious "comprehension gap." The bug was discovered through manual CoT inspection: the model's reasoning chains clearly stated correct ON decisions, but the parser recorded them as 0. All results in this paper use the corrected parser. The raw text of all 750 responses is saved for independent verification. The lesson: parser correctness must be validated against manually verified ground-truth responses before reporting evaluation results, especially when the parser output is used to draw qualitative conclusions about model capability.

---

9. Conclusion

We have presented the first verifiable RLVR environment for financial planning. The environment is grounded in the strategy-relative regime switching framework of Bilokon (2026), where Bellman backward induction provides exact optimal solutions against which LLM outputs can be scored.

Our parameter landscape analysis across 1,500 configurations fully characterizes when planning matters: signal persistence (κ) is the dominant parameter, the λ/α ratio has a non-monotone effect peaking at intermediate values, and the maximum disagreement between optimal and greedy policies is bounded at approximately 22% of states.

The planning mechanism is transparent and verifiable: the optimal policy is more aggressive than greedy because it amortizes switching costs over persistent future gains. The disagreement zone ($z \in [0.22, 0.50]$ for OFF $\rightarrow$ ON switching at our evaluation configuration) can be derived from first principles and verified against the Bellman Q-values.

Evaluating Claude Opus 4, we find that the model significantly outperforms random but significantly underperforms greedy. The model achieves near-perfect comprehension (99.3% easy accuracy), confirming it correctly understands payoff structure. On the 68 hard decisions where planning matters, Opus matches the greedy policy 94.1% of the time and the optimal policy only 5.9% of the time; CoT inspection confirms genuine planning in only 2/68 decisions (2.9%). There is

a single gap: the planning gap. Forward-looking financial planning is not an emergent capability of current frontier LLMs.

The two-bucket evaluation methodology extracts maximum diagnostic information from limited API budgets by separating comprehension from planning. We hope this approach proves useful beyond our specific gym.

The gym is open-source, with single-function TRL integration (`reward_fn(completions, problems) -> scores`), and is designed for extension to additional problem types within the strategy-relative framework. We provide it as infrastructure for the community to investigate whether RLVR training can elicit forward-looking financial reasoning in LLMs.

---

10. Future Work

Several directions extend naturally from this work.

RLVR training experiments. The most immediate next step is to run GRPO training (Shao et al., 2024) on the gym and measure whether planning capability improves. The two-bucket methodology provides a natural before/after metric: does hard-decision accuracy increase with training? We provide the reward function; the training infrastructure (TRL, veRL) is standard.

Suite expansion. The base class architecture supports additional financial planning problems with computable optimal solutions. Three candidates are immediate:

- *Almgren-Chriss optimal execution* (Almgren and Chriss, 2001): partition a large order across time periods, minimizing market impact plus risk. Closed-form solution under linear impact. Action space is continuous (trade size), testing a different reasoning mode.
- *Merton portfolio allocation* (Merton, 1969): allocate between risky and risk-free assets to maximize expected utility of terminal wealth. Closed-form for log and power utility. Tests risk-return reasoning.
- *Black-Scholes hedging with transaction costs* (Buehler et al., 2019): adjust hedge ratios optimally under proportional costs. Semi-analytical solutions via no-trade bands. Tests precision-versus-cost reasoning.

Each additional problem type adds structural diversity to the suite, addressing the single-problem limitation of the current gym.

Multi-dimensional signals. The paper's framework (Bilokon, 2026) allows Z_t to take values in a Polish space E, which includes $\mathbb{R}^n$. Extending from a scalar signal to a vector of market features (momentum, volatility, volume) would increase problem complexity while remaining within the theoretical framework. The Bellman solver would require a multi-dimensional grid or function approximation, increasing computational cost but not changing the verification principle.

Hidden regime detection (v2). A more challenging and epistemically richer extension is to make κ itself latent. At the start of each episode, κ is drawn from a binary set $\{\kappa_{\text{low}}, \kappa_{\text{high}}\}$ (e.g., $\{0.1, 0.7\}$) with known prior probability, but the agent is not told which regime it is in. The

agent must infer the current κ from the observed signal history and simultaneously plan over the switching decisions. The optimal policy is computed via a POMDP with belief state over κ — a distribution that is updated by Bayesian filtering at each step. This formulation addresses the asymmetric information limitation of the current gym (Section 8) by requiring the optimal agent to also infer persistence from data, making the comparison fair. More importantly, it raises the cognitive bar: a correct response requires (i) learning the signal's autocorrelation structure from the episode history, (ii) maintaining and updating a belief over regimes, and (iii) translating that belief into a forward-looking switching decision. This is much harder to dismiss as "just math" — the model must reason about model uncertainty, not merely about known-parameter optimization. The v2 gym is directly motivated by the finding that Opus fails on 95% of hard decisions even at $t = 11$, when the signal history is sufficient to distinguish κ_{low} from κ_{high} with high confidence. The failure is in inference and planning jointly, and a POMDP formulation directly isolates this capability.

Non-stationary dynamics. Real financial markets exhibit regime changes, structural breaks, and time-varying parameters. Extending the OU process to allow time-varying κ_t or $\sigma_{z,t}$ would test planning under non-stationarity. The Bellman recursion remains valid for time-inhomogeneous Markov processes; only the transition kernel changes at each step.

Model scaling analysis. We evaluated only Claude Opus 4. A systematic comparison across model sizes (Haiku, Sonnet, Opus) and model families (GPT-4, Gemini, open-weight models) would establish whether planning capability scales with model size or is a qualitative gap.

Prompt optimization. Our finding that prompt rephrasing improved $T = 1$ accuracy from 20% to 80% suggests that prompt quality is a major confound. Systematic prompt optimization (or the use of few-shot examples showing optimal reasoning) could disentangle prompt effects from genuine reasoning limitations.

Transfer to real financial data. The current gym uses synthetic data with known dynamics. Testing whether models trained on the gym exhibit improved decision-making on real (out-of-distribution) financial data would validate the transfer hypothesis. This requires careful experimental design to avoid confounding with data snooping.

References

Almgren, R. and Chriss, N. (2001). Optimal execution of portfolio transactions. *Journal of Risk*, 3(2):5–39.

Bilokon, P. (2026). Strategy-relative market regimes as filtration compressions. SSRN Working Paper 6504227.

Bilokon, P., Jacquier, A., and McIndoe, C. (2021). Market regime classification with signatures. arXiv preprint arXiv:2107.00066.

Buehler, H., Gonon, L., Teichmann, J., and Wood, B. (2019). Deep hedging. *Quantitative Finance*, 19(8):1271–1291.

DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. (2025). DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv preprint arXiv:2501.12948.

Dixon, M. F., Halperin, I., and Bilokon, P. (2020). *Machine Learning in Finance: From Theory to Practice*. Springer.

Hambly, B., Xu, R., and Yang, H. (2023). Recent advances in reinforcement learning in finance. *Mathematical Finance*, 33(3):437–503.

Halperin, I. (2020). The QLBS Q-Learner in the Black-Scholes(-Merton) world. *Journal of Derivatives*, 28(1):99–122.

Liu, X.-Y., Xia, Z., Rui, J., Gao, J., Yang, H., Zhu, M., Wang, C., Wang, Z., and Guo, J. (2022). FinRL-Meta: Market environments and benchmarks for data-driven financial reinforcement learning. *NeurIPS 2022 Datasets and Benchmarks Track*.

Liu, Y., Guo, Z., et al. (2025). Fin-R1: A large language model for financial reasoning through reinforcement learning. arXiv preprint arXiv:2503.16252.

Merton, R. C. (1969). Lifetime portfolio selection under uncertainty: The continuous-time case. *The Review of Economics and Statistics*, 51(3):247–257.

Selig, E. and Bilokon, P. (2024). Regularised jump models for regime identification and feature selection. SSRN Working Paper 4950423.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y., Wu, Y., and Guo, D. (2024). DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300.

Songtao, L., et al. (2025). Crossing the reward bridge: Expanding RL with verifiable rewards across diverse domains. arXiv preprint arXiv:2503.23829.

Stojanovski, D., et al. (2025). Reasoning Gym: Reasoning environments for reinforcement learning with verifiable rewards. *NeurIPS 2025 Spotlight*. arXiv preprint arXiv:2505.24760.

Wang, B., Gao, X., and Li, L. (2026). Reinforcement learning for continuous-time optimal execution: Actor-critic algorithm and error analysis. *Finance and Stochastics*.

Appendix

A. Top Configurations by Disagreement

The 10 highest-disagreement configurations from the 1,500-configuration sweep:

Rank	T	κ	λ	α	σ_z	λ/α	Disagree (%)	J_{opt}	J_{greedy}	J_{random}	J-Capture (%)
1	25	0.1	0.20	0.15	0.3	1.33	21.97	0.69	0.36	−2.38	52.7
2	25	0.1	0.05	0.10	0.1	0.50	21.82	0.17	0.08	−0.53	46.0

Rank	T	κ	λ	α	σ_z	λ/α	Disagree (%)	J_{opt}	J_{greedy}	J_{random}	J-Capture (%)
3	25	0.1	0.10	0.10	0.2	1.00	21.82	0.31	0.13	−1.15	41.9
4	25	0.1	0.10	0.20	0.1	0.50	21.82	0.31	0.13	−1.15	41.9
5	25	0.1	0.15	0.10	0.3	1.50	21.82	0.43	0.22	−1.80	44.2

All top configurations share $\kappa = 0.1$ (maximum persistence) and $T = 25$ (longest horizon), confirming the parameter landscape analysis of Section 4.

B. Opus Evaluation: Aggregate Statistics

All figures use the corrected parser. The buggy-parser figures (Easy 77.4%, Hard 4.8%) are superseded and must not be cited.

Metric	Value (corrected parser)
Total instances	30
Total decisions	750
Easy decisions	682
Hard decisions	68
Easy accuracy	99.3% (677/682)
Hard: matches optimal	5.9% (4/68)
Hard: matches greedy	94.1% (64/68)
Hard: matches neither	0.0% (0/68)
CoT-verified genuine planning	2.9% (2/68)
CoT-verified borderline	2.9% (2/68)
Mean J (Opus)	−0.16
Mean J (Optimal)	+1.30
Mean J (Greedy)	+1.19
Mean J (Random)	−1.81

Parser bug note. The original evaluation (test 10 in Appendix F) used a parser that over-matched step identifiers and failed to capture `s_t = 1` correctly. This produced Easy 77.4% (484/625) and Hard 4.8% (6/125) — all wrong. Test 17 (corrected parser) supersedes test 10.

C. Solver Verification at $T = 1$

At $T = 1$, the Bellman solution reduces to greedy because there is no future to plan for. The optimal decision is:

$$s_0^* = \begin{cases} 1 & \text{if } \alpha \cdot Z_0 > \lambda \\ 0 & \text{otherwise} \end{cases}$$

We verified that the numerical solver reproduces this closed-form solution exactly for all grid points, confirming correctness of the backward induction implementation.

D. Threshold Verification at $t = 7$, $z = 0.35$

For the evaluation configuration ($\kappa = 0.1$, $\lambda = 0.15$, $\alpha = 0.30$), the greedy agent stays OFF at $z = 0.35$ because $\alpha \cdot z = 0.105 < 0.15 = \lambda$. The Bellman Q-values at $(t = 7, z = 0.35, s^- = 0)$ are:

$$Q(z, a{=}0, s^-{=}0) < Q(z, a{=}1, s^-{=}0)$$

confirming that the optimal policy switches ON. The difference $Q(a{=}1) - Q(a{=}0) > 0$ reflects the cumulative expected gain from being ON during the persistent positive signal regime, net of the one-time switching cost.

E. Prompt Versions

Old prompt (broken): Achieved approximately 20% accuracy at $T = 1$, indicating that the model could not correctly evaluate immediate payoffs due to ambiguous problem framing.

Improved prompt: Explicitly states the payoff structure — that being ON yields PnL of $\alpha \cdot Z_t$ and being OFF yields 0, with switching cost λ incurred only on changes. Achieved approximately 80% accuracy at $T = 1$, confirming that the comprehension gap was partially due to prompt ambiguity.

The improved prompt was used for all Opus results reported in this paper. The fact that a 4x accuracy improvement was achieved through prompt rephrasing alone underscores the importance of prompt design in RLVR evaluation.

Figures

Appendix F: Complete Test Summary

All tests conducted on Claude Opus 4 (`claude-opus-4-20250514`) or gym infrastructure. Prompt versions: **N/A** = solver/infrastructure (no LLM); **OLD** = broken prompt (no payoff explanation); **IMPROVED** = explicit RULES with payoff structure; **SCAFFOLD** = improved + pre-computed options + trend hints; **IMPROVED*** = improved but single-turn (future info leaked). Tests marked † were superseded or confounded and are excluded from main results.

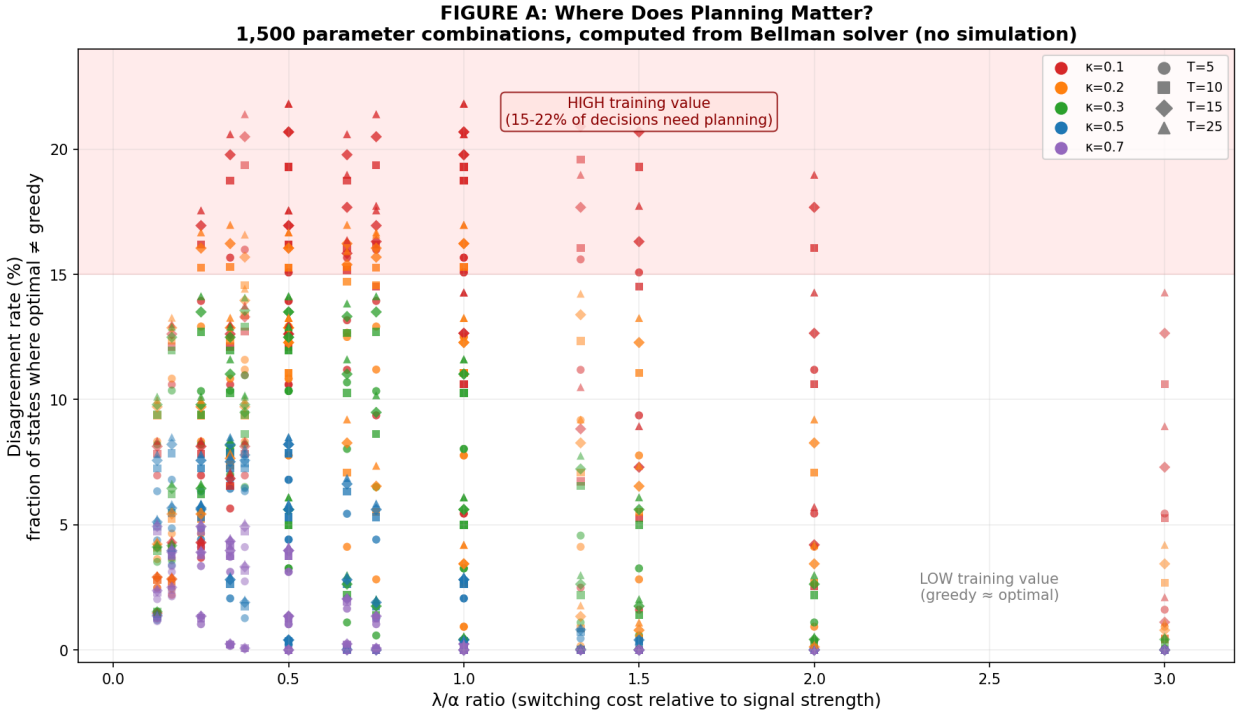


Figure 1: Figure A — Where Planning Matters (1,500 configurations)

Test #	Description	Prompt Version	Config	N	Key Result	Confidence	Data File
1	Solver correctness: T=1 reduces to greedy	N/A	κ=0.1, λ=0.15, α=0.30, T=1	All grid points	Solver reproduces closed-form exactly	High	—
2	Goldilocks: easy level	N/A	κ=0.1, λ=0.05, α=0.30, T=10	N=500	optimal > greedy > random; greedy capture ~98%	High	—
3	Goldilocks: medium level	N/A	κ=0.1, λ=0.15, α=0.30, T=15	N=500	optimal > greedy > random; greedy capture ~90%	High	—

Test #	Description	Prompt Version	Config	N	Key Result	Confidence	Data File
4	Goldilocks: hard level	N/A	κ=0.1, λ=0.15, α=0.30, T=25	N=500	0-capture drops to 40–60%; accuracy stays ~90%	High	—
5	Parameter sweep	N/A	1,500 (κ,λ,α,σz,T) combinations	1,500 configs	κ dominant; max disagreement ~22%; λ/α peaks at 0.33–0.50	High	full_parameter_sweep.json
6	T=1 accuracy, old prompt	OLD	κ=0.1, λ=0.15, α=0.30, T=1	Small	~20% accuracy — prompt comprehension failure	High	—
7†	Early multi-turn eval, old prompt	OLD	Various	Various	Superseded/confounded — excluded from main results	Low	—
8†	Sonnet/Haiku eval, old prompt	OLD	Various	Various	Superseded/confounded — excluded from main results	Low	—
9	T=1 accuracy, improved prompt	IMPROVED	κ=0.1, λ=0.15, α=0.30, T=1	Small	~80% accuracy — prompt fix confirmed	High	—

Test #	Description	Prompt Version	Config	N	Key Result	Confidence	Data File
10†	Main Opus API eval (30 instances) — SUPERSEDED by test 17 (buggy parser)	IMPROVED	κ=0.1, λ=0.15, α=0.30, T=25	N=30 (750 decisions)	BUGGY PARSER: Easy 77.4% (484/625); Hard 4.8% (6/125) — all numbers wrong; do not cite	Low	eval_llm_results.json (raw text archived)
11	Scaffolded manual test (seed=47)	SCAFFOLD	κ=0.1, λ=0.15, α=0.30, T=25	N=1 (25 decisions)	25/25 correct incl. 1 hard; scaffold did the planning	Medium	—
12	Minimal-prompt manual test (seed=49)	IMPROVED	κ=0.1, λ=0.15, α=0.30, T=10	N=1 (10 decisions)	9/9 easy (100%); 0/1 hard; greedy reasoning confirmed	Medium	—
13	Hard-decision deep-dive (seed=21)	IMPROVED	κ=0.1, λ=0.15, α=0.30, T=25	N=1 (25 decisions)	9 hard decisions; model follows greedy throughout	Medium	—
14†	Single-turn eval (future info leaked)	IMPROVED*	κ=0.1, λ=0.15, α=0.30, T=25	Various	Superseded/confounded — excluded from main results	Low	—

Test #	Description	Prompt Version	Config	N	Key Result	Confidence	Data File
15†	Confounded multi-turn variant	IMPROVED*	Various	Various	Superseded/confounded — excluded from main results	Low	—
16	Kappa inference test (κ=0.1 vs κ=0.7)	IMPROVED	α=0.30, λ=0.15, T=15; κ varied	N=5 per condition	Identical behavior (95.9% easy, 0.4 switches); only 3 hard decisions — inconclusive	Low	eval_opus_buckets.json
17	Main Opus API eval — corrected parser	IMPROVED	κ=0.1, λ=0.15, α=0.30, T=25	N=30 (750 decisions: 682 easy, 68 hard)	Easy 99.3% (677/682); Hard 5.9% (4/68); CoT: 2/68 genuine planning, 2/68 borderline	High	eval_llm_results.json (raw text re-parsed)

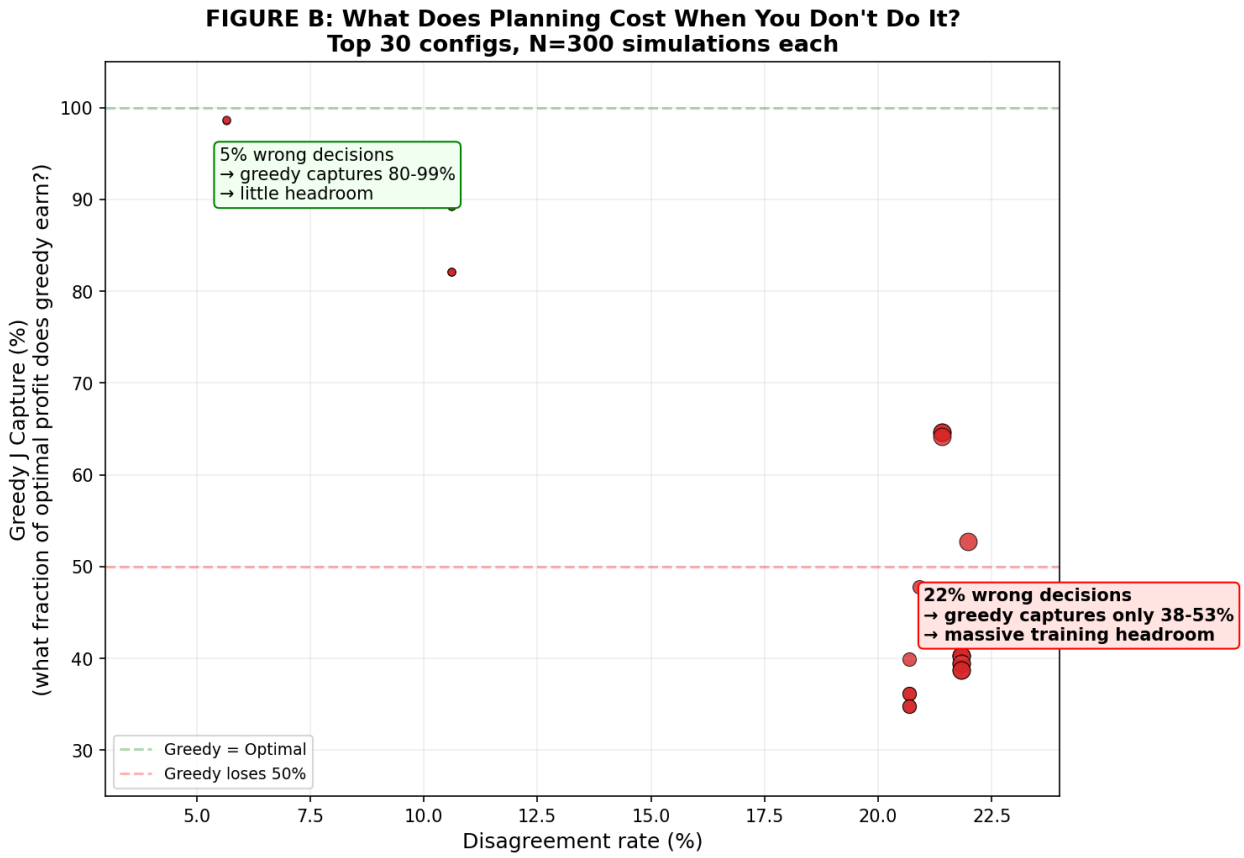


Figure 2: Figure B — Cost of Not Planning (disagreement vs J-capture)

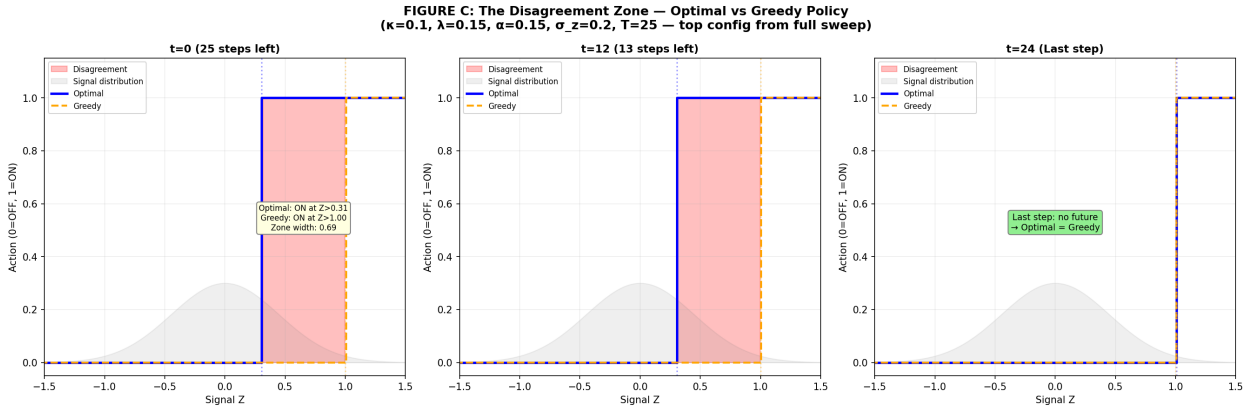


Figure 3: Figure C — The Disagreement Zone (policy comparison at 3 time steps)

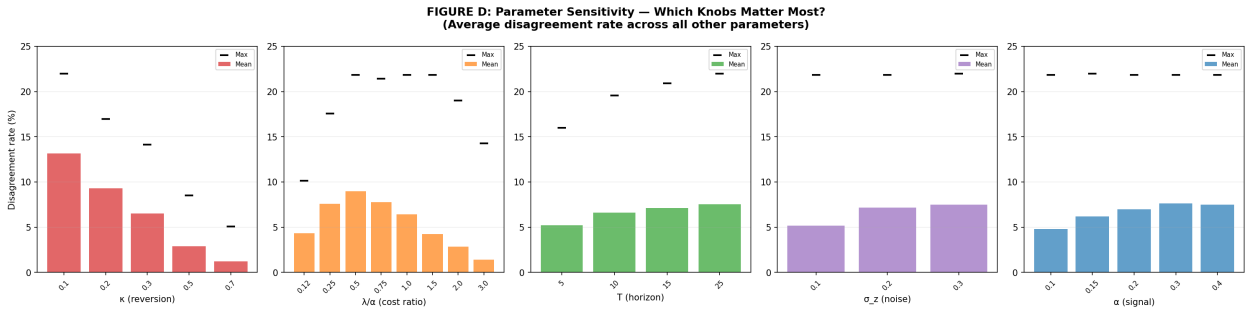


Figure 4: Figure D — Parameter Sensitivity (which knobs matter most)

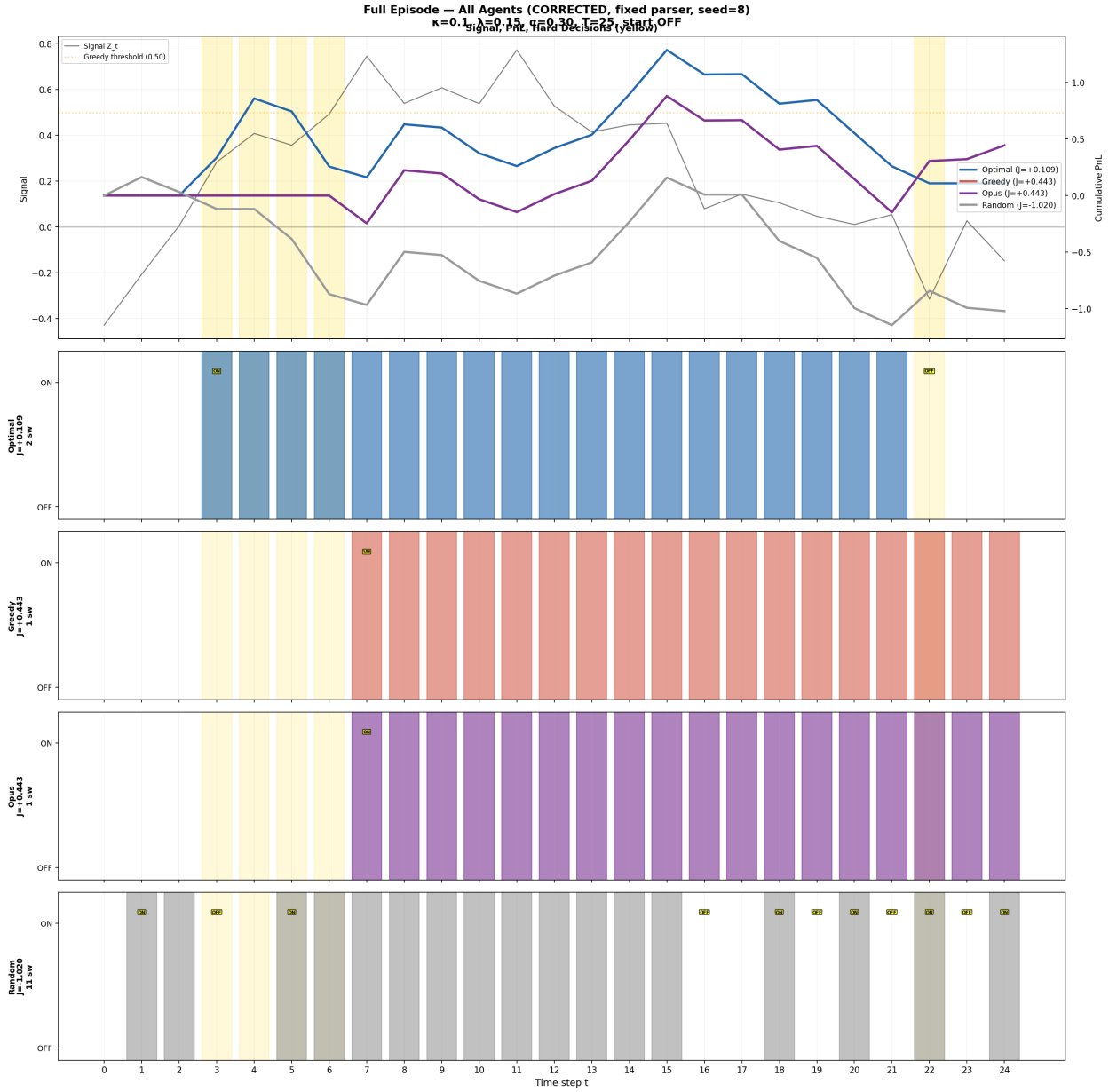


Figure 5: Figure E — Full Episode: Opus vs All Agents (seed=8, Opus alternating)

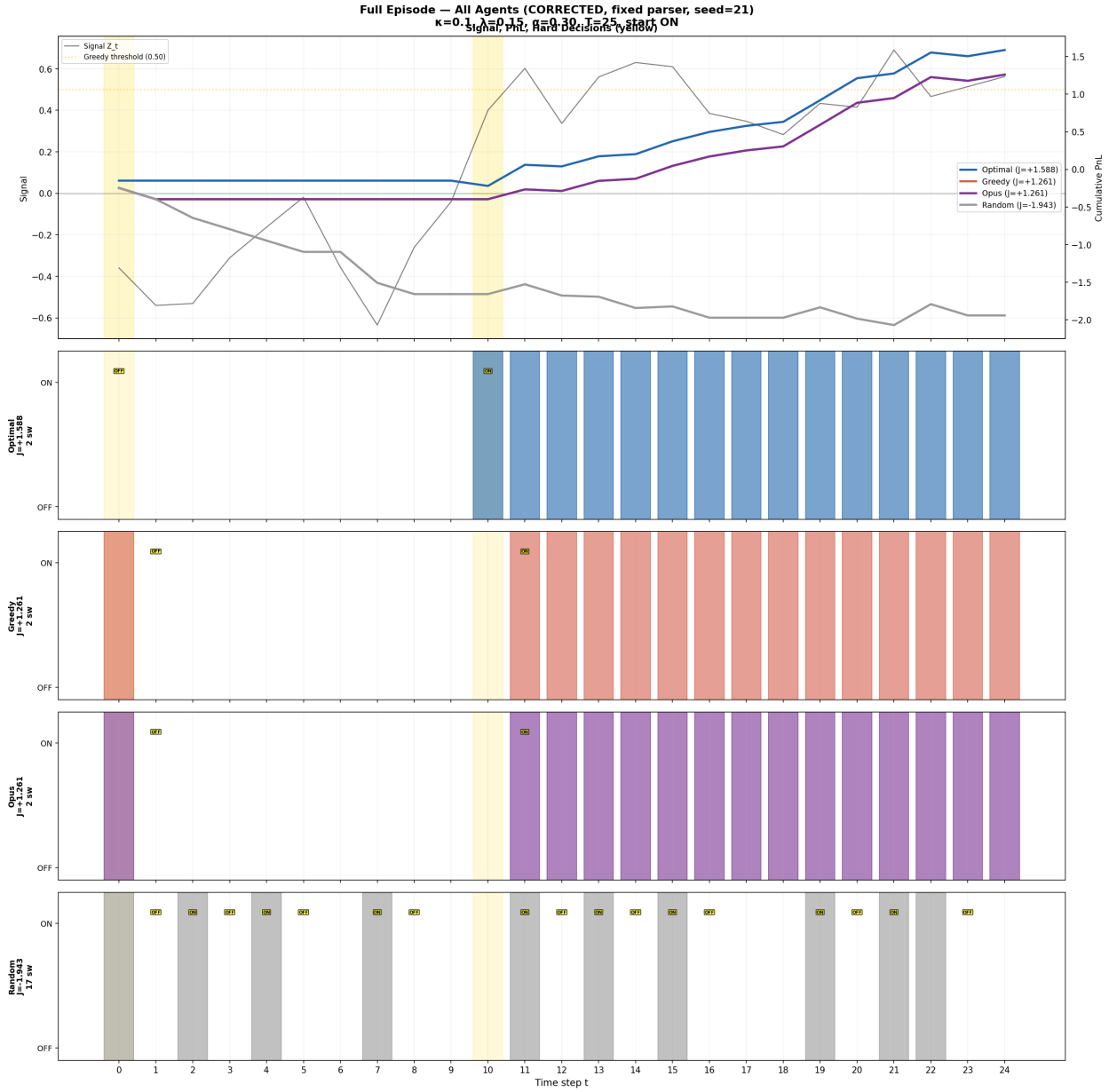


Figure 6: Figure F — Full Episode: 9 Hard Decisions (seed=21)

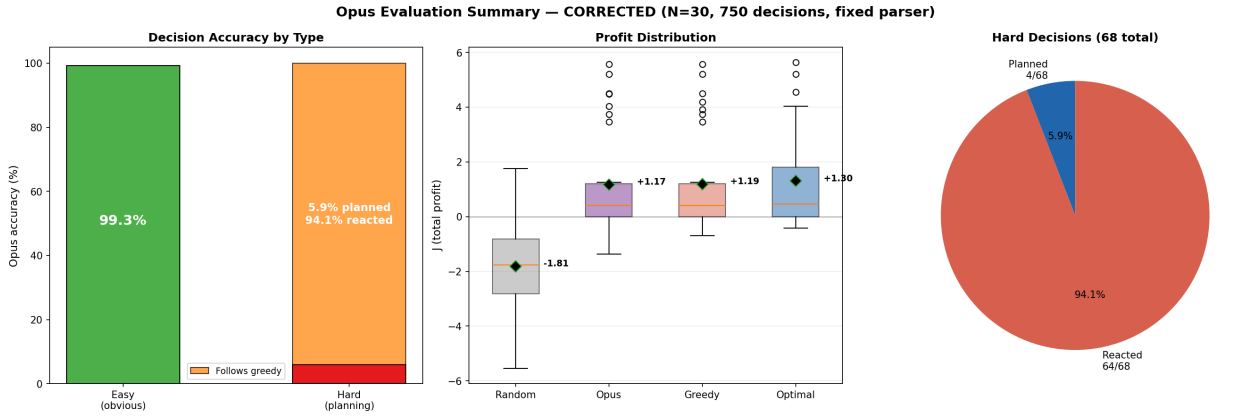


Figure 7: Figure G — Opus Evaluation Summary (3 panels)

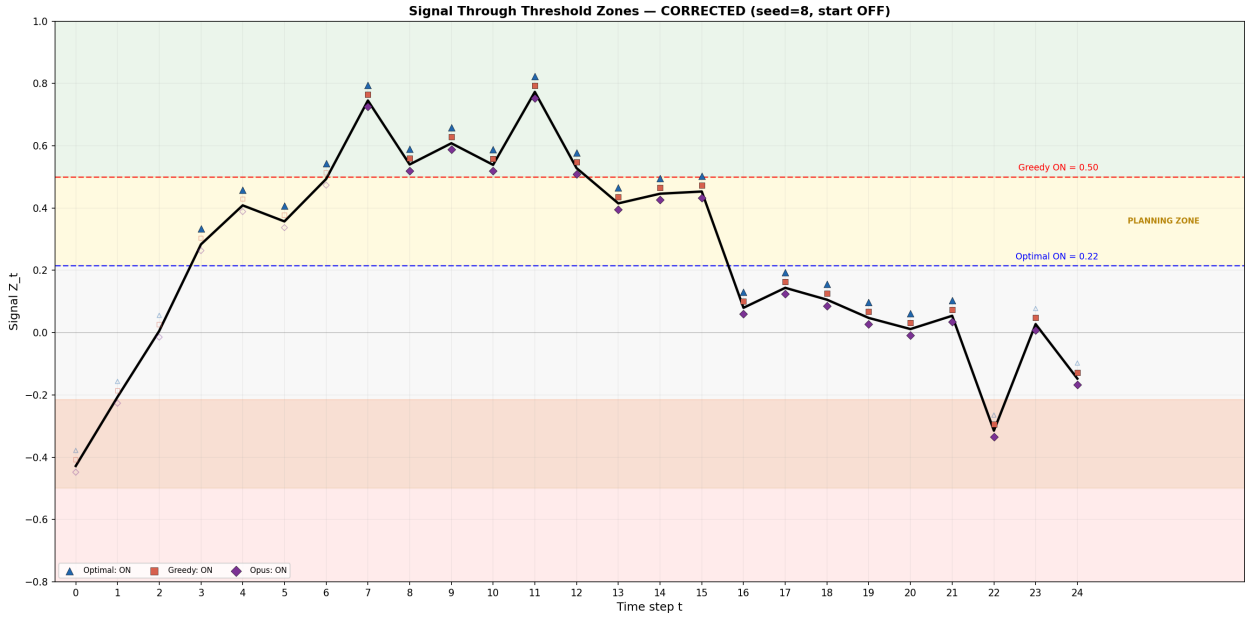


Figure 8: Figure H — Signal Path Through Threshold Zones

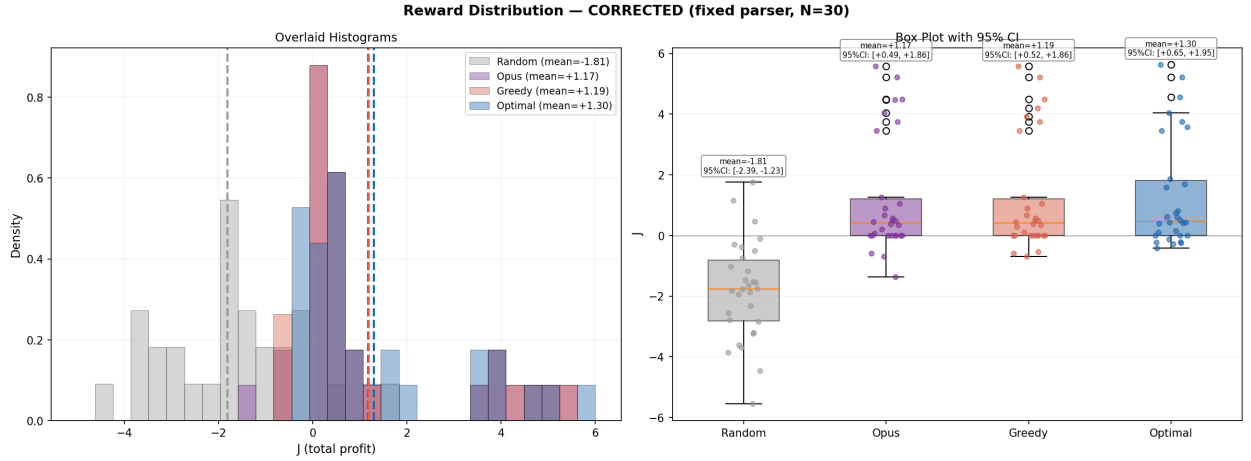


Figure 9: Figure I — Reward Distribution (histogram and box plot)

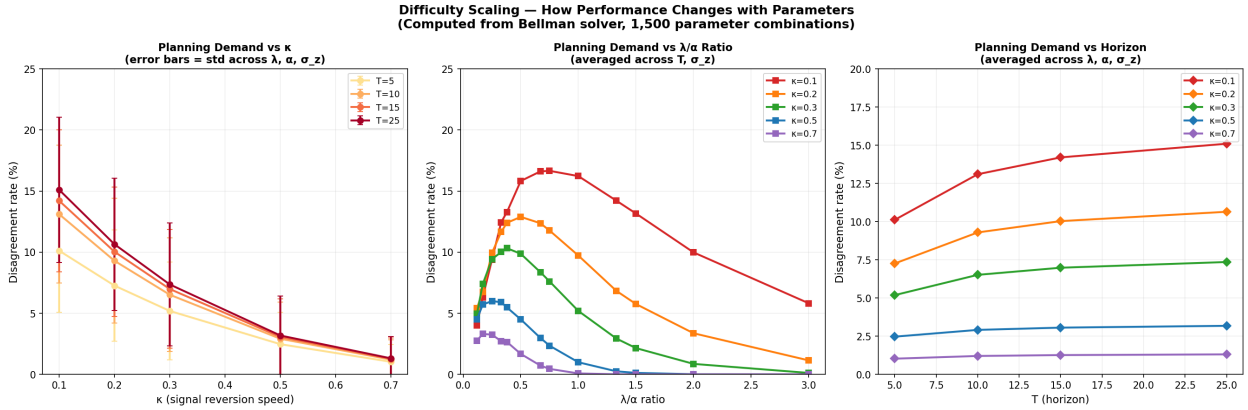
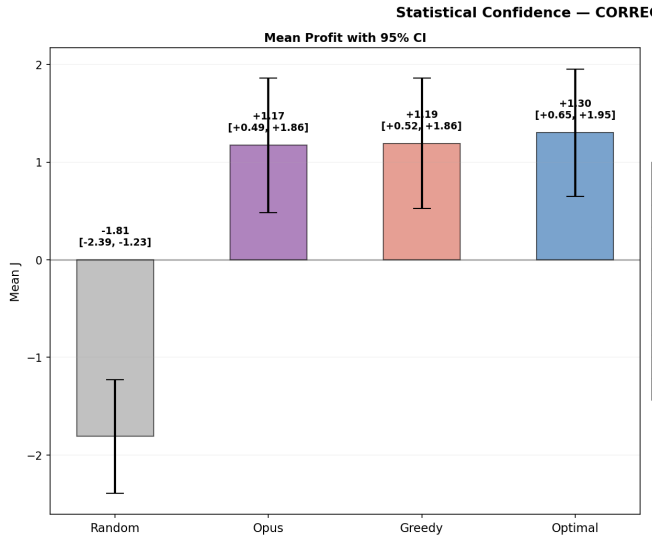


Figure 10: Figure J — Difficulty Scaling (performance vs parameters)



Statistical Tests (corrected)

Comparison	Mean diff	t-stat	p-value	Significant?
Opus vs Random	+2.984	6.49	0.0000	YES
Opus vs Greedy	-0.018	-0.04	0.9708	no
Opus vs Optimal	-0.127	-0.26	0.7936	no
Greedy vs Optimal	-0.109	-0.23	0.8195	no
CORRECTED RESULTS				
Easy accuracy	99.3%	677/682		
Hard: plans	5.9%	4/68		
Hard: greedy	94.1%	64/68		
CoT-verified	2.9%	2/68		

Figure 11: Figure K — Statistical Confidence (CIs and t-tests)